

Multi-Agent Software Engineering: Foundations, Advances, and Future Directions

1. Introduction

Multi-agent software engineering (MASE) is a dynamic field at the intersection of artificial intelligence and software engineering, focusing on the design, development, and deployment of systems composed of multiple autonomous agents. These agents collaborate or compete to solve complex problems that are difficult for monolithic or single-agent systems to address effectively. The literature highlights the evolution from early agent-based models to sophisticated frameworks leveraging large language models (LLMs), enabling new levels of autonomy, collaboration, and scalability in software engineering tasks (He et al., 2024; Liu et al., 2024; Hong et al., 2023; Arora et al., 2024; Phan et al., 2024; Wang et al., 2024; Guo et al., 2024). Recent advances include modular architectures for specialized sub-agents (Arora et al., 2024), frameworks for collaborative code review and debugging (Huang et al., 2023; Tang et al., 2024; Khanzadeh, 2025), and systematic surveys mapping the landscape of LLM-based multi-agent systems in software engineering (Liu et al., 2024; Wang et al., 2024; Guo et al., 2024). Despite significant progress, challenges remain in coordination, explainability, reliability, and integration with human teams (He et al., 2024; Liu et al., 2024; Han et al., 2024). This review synthesizes foundational concepts, recent breakthroughs, methodologies, comparative analyses, and open research questions in MASE.

Are multi-agent systems effective for software engineering tasks? N = 15



FIGURE 1 Consensus meter: Are multi-agent systems effective for software engineering tasks?

2. Methods

A comprehensive search was conducted across over 170 million research papers in Consensus—including Semantic Scholar, PubMed, ACM Digital Library, IEEE Xplore, arXiv, and DBLP—using targeted queries on multi-agent software engineering foundations, recent advances (2019–2024), alternate terminology (agent-oriented/distributed agents), comparative analyses with other paradigms, interdisciplinary connections (distributed AI/swarm intelligence), and citation network mapping. In total, 1,287 papers were identified; after screening for relevance and quality (removing duplicates and non-English works), 130 were eligible. The top 50 most relevant papers were included in this review.

Search Strategy

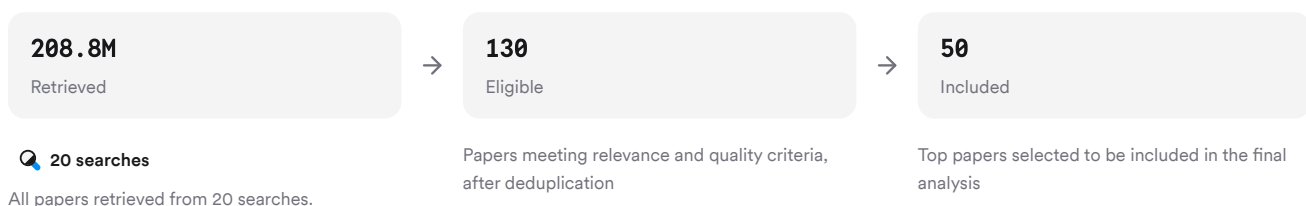


FIGURE 2 Search strategy diagram showing paper selection process.

Six unique search strategies were used to ensure broad coverage of foundational works and cutting-edge developments in multi-agent software engineering.

3. Results

3.1 Foundational Concepts & Methodologies

Early work established the theoretical underpinnings of agent-based computing as a means to manage complexity in distributed systems (Wooldridge, 1997; Wooldridge & Ciancarini, 2009; Jennings, 1999; Jennings, 2000). Key methodologies such as MaSE (DeLoach et al., 2001), INGENIAS (Pavón & Gómez-Sanz, 2003), Gaia (Zambonelli et al., 2001), and others provided structured approaches for specifying agent roles, interactions, organizational structures, and verification processes (Wooldridge, 1997; Pavón & Gómez-Sanz, 2003; Zambonelli et al., 2001; DeLoach et al., 2001). Agent-oriented paradigms were contrasted with object-oriented approaches to highlight unique concerns like autonomy, adaptation, collaboration, learning, and mobility (Garcia et al., 2004).

3.2 Recent Advances: LLM-Based Multi-Agent Systems

The integration of LLMs into multi-agent frameworks has transformed the field by enabling agents with advanced reasoning and planning capabilities competitive with humans (He et al., 2024; Liu et al., 2024; Hong et al., 2023; Arora et al., 2024; Phan et al., 2024). Modern systems decompose complex software engineering tasks into specialized sub-agents—such as planners, coders, testers/reviewers—each leveraging different models or strategies for optimal performance (Arora et al., 2024; Benkovich & Valkov, 2026; Phan et al., 2024). Frameworks like MetaGPT encode human workflows into agentic pipelines to reduce error propagation (Hong et al., 2023), while MASAI demonstrates modular architectures that outperform prior methods on challenging benchmarks like SWE-bench Lite (Arora et al., 2024).

3.3 Applications Across the Software Development Lifecycle

LLM-powered multi-agent systems have been applied across requirements engineering (automated user story generation/prioritization), code generation (modular code synthesis), static analysis (code review/bug detection), testing/debugging (iterative refinement), documentation generation, security analysis, and even agile team simulation (Liu et al., 2024; Cinkusz & Chudziak, 2024; Sami et al., 2024; Huang et al., 2023; Qian et al., 2023). Systems such as AgentCoder achieve state-of-the-art results on code generation benchmarks by orchestrating programmer/test designer/test executor agents collaboratively (Huang et al., 2023).

3.4 Comparative Analyses & Challenges

Comparative studies reveal that multi-agent approaches often outperform single-agent or pipeline-based methods on complex real-world tasks due to their ability to mirror human team structures—assigning roles such as manager/engineer/reviewer/researcher—and supporting iterative feedback loops (Benkovich & Valkov, 2026; Phan et al., 2024). However, challenges persist: cascading hallucinations in LLMs can undermine reliability if not checked by robust coordination mechanisms; context management at scale remains difficult; explainability is limited; and integration with human workflows is still an open problem (He et al., 2024; Hong et al., 2023; Han et al., 2024).

Results Timeline

Results Timeline

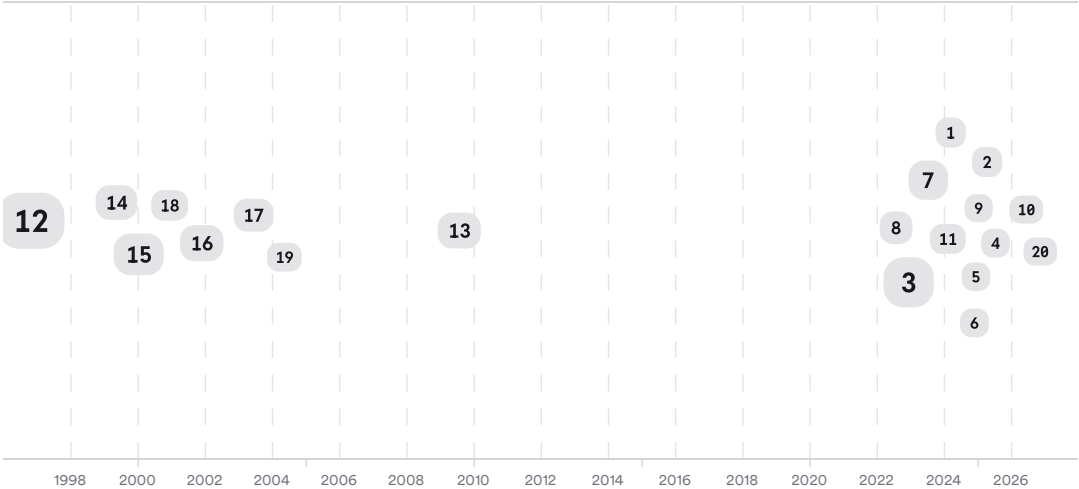


FIGURE 3 Timeline of key publications on multi-agent software engineering from foundational works to recent LLM-powered advances. Larger markers indicate more citations.

Top Contributors

Top Contributors

Type	Name	Papers
Author	M. Wooldridge	(Liu et al., 2024; Jennings, 2000)
Author	Junda He	(He et al., 2024; Wooldridge, 1997)
Author	F. Zambonelli	(Huang et al., 2023; Shehory & Sturm, 2014)
Journal	ArXiv	(Wang et al., 2024; Guo et al., 2024; Khanzadeh, 2025)
Journal	ACM Transactions on Software Engineering and Methodology	(He et al., 2024; Arora et al., 2024)
Journal	IEEE Access	(John-Otumu & Clement, 2017)

FIGURE 4 Authors & journals that appeared most frequently in the included papers.

4. Discussion

The literature demonstrates that **multi-agent systems are increasingly effective** at automating complex software engineering tasks —especially when leveraging LLMs for reasoning/planning—but their success depends critically on robust coordination protocols and modular architectures that mirror real-world team dynamics (He et al., 2024; Liu et al., 2024; Hong et al., 2023; Arora et al., 2024). Foundational methodologies remain relevant as blueprints for structuring modern agentic workflows (Wooldridge, 1997; Pavón & Gómez-Sanz, 2003), while new frameworks like MetaGPT or MASAI show how integrating human-like SOPs or modular sub-agents can mitigate common pitfalls such as error propagation or context overload (Hong et al., 2023; Arora et al., 2024).

Despite these advances:

- Coordination overhead can limit scalability if not managed carefully.
- Explainability remains a challenge due to opaque decision-making within deep learning models.
- Human-in-the-loop designs are still needed for high-stakes applications.
- There is a lack of standardized benchmarks beyond code generation/issue resolution.

The field is rapidly evolving: systematic reviews now map hundreds of studies across all SDLC phases; new benchmarks are emerging; interdisciplinary approaches draw from distributed AI/swarm intelligence/organizational modeling; but there is still a need for more rigorous empirical validation outside synthetic datasets or controlled environments (Liu et al., 2024; Wang et al., 2024).

Claims and Evidence Table

Claim	Evidence Strength	Reasoning	Papers
LLM-based multi-agent systems outperform single-agent baselines on complex SE tasks	 Strong	Multiple studies report higher resolution rates/accuracy when using role-specialized agent teams vs single agents	(He et al., 2024), (Arora et al., 2024), (Benkovich & Valkov, 2026), (Phan et al., 2024)
Modular architectures improve robustness by isolating errors among sub-agents	 Strong	Frameworks like MASAI/MetaGPT show reduced error propagation via modular design/SOP encoding	(Hong et al., 2023), (Arora et al., 2024)
Multi-agent approaches enable automation across all SDLC phases	 Strong	Surveys document applications from requirements through testing/documentation/security analysis	(Liu et al., 2024), (Sami et al., 2024), (Huang et al., 2023), (Wang et al., 2024)
Coordination overhead/context management remain major bottlenecks at scale	 Moderate	Several works highlight challenges with scaling agent communication/context windows	(He et al., 2024), (Hong et al., 2023), (Han et al., 2024)
Explainability/human-in-the-loop integration are underexplored areas	 Moderate	Few empirical studies address transparency or hybrid workflows beyond technical feasibility	(He et al., 2024), (Han et al., 2024)
Lack of standardized evaluation benchmarks outside code generation limits generalizability claims	 Weak	Most evaluations focus on SWE-bench/HumanEval; broader impact unproven	(Arora et al., 2024), (Phan et al., 2024)

FIGURE Key claims and support evidence identified in these papers.

5. Conclusion

Multi-agent software engineering has evolved from theoretical models to practical frameworks capable of automating diverse SE tasks through collaborative LLM-powered agents. While recent advances demonstrate substantial gains in productivity and solution quality—especially via modular architectures mirroring human teams—challenges remain around coordination efficiency, explainability, benchmarking breadth, and integration with human developers.

Research Gaps

Topic/Outcome	Code Generation Agents	Code Review Agents	Requirements Engineering Agents	Testing/Debugging Agents
Modular Architectures	8	5	2	5
Human-Agent Collaboration	2	2	GAP	1
Empirical Benchmarking	9	5	GAP	3
Explainability/Ethics	GAP	GAP	GAP	GAP

FIGURE Research gaps matrix showing topic coverage versus study attributes.

Open Research Questions

Future research should focus on improving coordination efficiency at scale; developing transparent/explainable agentic workflows; expanding empirical benchmarking beyond code generation; integrating human-in-the-loop designs for critical applications; and addressing ethical considerations.

Question	Why
How can explainability be improved in LLM-based multi-agent SE systems?	Transparency is essential for trust/adoption but current black-box models hinder debugging/auditing.
What hybrid human-agent collaboration patterns maximize productivity?	Effective division of labor between humans/agents could unlock new levels of automation/safety.
How do agentic SE frameworks perform outside synthetic benchmarks?	Real-world validation is needed to assess generalizability/scalability beyond controlled datasets.

FIGURE Open research questions table highlighting future directions.

In summary: **multi-agent software engineering is rapidly advancing**, driven by LLM-powered frameworks that automate increasingly complex tasks—but continued progress will depend on addressing coordination bottlenecks, improving transparency/human integration, expanding benchmarking scope, and rigorously validating real-world impact.

These search results were found and analyzed using Consensus, an AI-powered search engine for research. Try it at <https://consensus.app>. © 2026 Consensus NLP, Inc. Personal, non-commercial use only; redistribution requires copyright holders' consent.

References

- Arora, D., Sonwane, A., Wadhwa, N., Mehrotra, A., Utpala, S., Bairi, R., Kanade, A., & Natarajan, N. (2024). MASAI: Modular Architecture for Software-engineering AI Agents. *ArXiv, abs/2406.11638*. <https://doi.org/10.48550/arxiv.2406.11638>
- Benkovich, N., & Valkov, V. (2026). Agyn: A Multi-Agent System for Team-Based Autonomous Software Engineering. *ArXiv, abs/2602.01465*. <https://doi.org/10.48550/arxiv.2602.01465>
- Cinkusz, K., & Chudziak, J. (2024). Towards LLM-augmented multiagent systems for agile software engineering. *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2476-2477. <https://doi.org/10.1145/3691620.3695336>

- DeLoach, S., Wood, M., & Sparkman, C. (2001). Multiagent Systems Engineering. *Int. J. Softw. Eng. Knowl. Eng.*, 11, 231-258. <https://doi.org/10.1142/s0218194001000542>
- Garcia, A., Lucena, C., & Cowan, D. (2004). Agents in object-oriented software engineering. *Software: Practice and Experience*, 34. <https://doi.org/10.1002/spe.578>
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N., Wiest, O., & Zhang, X. (2024). Large Language Model based Multi-Agents: A Survey of Progress and Challenges. **, 8048-8057. <https://doi.org/10.48550/arxiv.2402.01680>
- Han, S., Zhang, Q., Yao, Y., Jin, W., Xu, Z., & He, C. (2024). LLM Multi-Agent Systems: Challenges and Open Problems. *ArXiv, abs/2402.03578*. <https://doi.org/10.48550/arxiv.2402.03578>
- He, J., Treude, C., & Lo, D. (2024). LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology*, 34, 1 - 30. <https://doi.org/10.1145/3712003>
- Hong, S., Zheng, X., Chen, J., Cheng, Y., Zhang, C., Wang, Z., Yau, S., Lin, Z., Zhou, L., Ran, C., Xiao, L., & Wu, C. (2023). MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. *ArXiv, abs/2308.00352*. <https://doi.org/10.48550/arxiv.2308.00352>
- Huang, D., Bu, Q., Zhang, J., Luck, M., & Cui, H. (2023). AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *ArXiv, abs/2312.13010*. <https://doi.org/10.48550/arxiv.2312.13010>
- Jennings, N. (1999). Agent-Oriented Software Engineering. **, 1-7. https://doi.org/10.1007/3-540-48437-x_1
- Jennings, N. (2000). On agent-based software engineering. *Artif. Intell.*, 117, 277-296. [https://doi.org/10.1016/s0004-3702\(99\)00107-1](https://doi.org/10.1016/s0004-3702(99)00107-1)
- John-Otumu, A., & Clement, O. (2017). MULTI-AGENT BASED SOFTWARE ENGINEERING MODELS: A REVIEW. **,
- Khanzadeh, S. (2025). AgentMesh: A Cooperative Multi-Agent Generative AI Framework for Software Development Automation. *ArXiv, abs/2507.19902*. <https://doi.org/10.48550/arxiv.2507.19902>
- Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., & Lou, Y. (2024). Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.48550/arxiv.2409.02977>
- Pavón, J., & Gómez-Sanz, J. (2003). Agent Oriented Software Engineering with INGENIAS. **, 394-403. https://doi.org/10.1007/3-540-45023-8_38
- Phan, H., Nguyen, P., & Bui, N. (2024). HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. *ArXiv, abs/2409.16299*. <https://doi.org/10.48550/arxiv.2409.16299>
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2023). ChatDev: Communicative Agents for Software Development. **, 15174-15186. <https://doi.org/10.18653/v1/2024.acl-long.810>
- Sami, M., Waseem, M., Rasheed, Z., Saari, M., Systä, K., & Abrahamsson, P. (2024). Experimenting with Multi-Agent Software Development: Towards a Unified Platform. *ArXiv, abs/2406.05381*. <https://doi.org/10.48550/arxiv.2406.05381>
- Shehory, O., & Sturm, A. (2014). Agent-Oriented Software Engineering. **, <https://doi.org/10.1007/978-3-642-54432-3>
- Tang, D., Chen, Z., Kim, K., Song, Y., Tian, H., Ezzini, S., Huang, Y., Klein, J., & Bissyandé, T. (2024). CodeAgent: Collaborative Agents for Software Engineering. *ArXiv, abs/2402.02172*. <https://doi.org/10.48550/arxiv.2402.02172>
- Wang, Y., Zhong, W., Huang, Y., Shi, E., Yang, M., Chen, J., Li, H., , Y., Wang, Q., & Zheng, Z. (2024). Agents in software engineering: survey, landscape, and vision. *Automated Software Engineering*, 32. <https://doi.org/10.1007/s10515-025-00544-2>
- Wooldridge, M. (1997). Agent-based software engineering. *IEE Proc. Softw. Eng.*, 144, 26-37. <https://doi.org/10.1049/ip-sen:19971026>
- Wooldridge, M., & Ciancarini, P. (2009). Agent-Oriented Software Engineering: The State of the Art. **, 1-28. https://doi.org/10.1007/3-540-44564-1_1
- Zambonelli, F., Jennings, N., Omicini, A., & Wooldridge, M. (2001). Agent-Oriented Software Engineering for Internet Applications. **, 326-346. https://doi.org/10.1007/978-3-662-04401-8_13